PRACTICAL 1: INTRODUCTION TO R / PYTHON ENVIRONMENT

Objective: To set up a statistical computing environment and practice basic syntax, data structures, and file I/O.

Question 1

Install Python/Jupyter (or R/RStudio) and verify the setup.

Procedure:

1. Download Python 3.x from python.org → install (tick "Add to PATH")
2. Verify: python --version
3. Install Jupyter: pip install notebook
4. Launch: jupyter notebook

(R alternative: install R from cran.r-project.org, then RStudio from posit.co)

Question 2

Practice basic syntax and operators.

Solution:

```
a, b = 15, 4
print("Addition:", a + b)
print("Division:", a / b)
print("Modulus:", a % b)
print("Power:", a ** 2)
print("a > b:", a > b)
```

Question 3

Create vectors, lists, and data frames.

Solution:

```
import numpy as np
import pandas as pd

scores = np.array([78, 85, 92, 67, 74, 88])
students = ["Aarav", "Priya", "Rohit", "Sneha", "Kunal", "Divya"]
df = pd.DataFrame({"Student": students, "Score": scores})
print(df)
```

Question 4

Read and write CSV files.

Solution:

```
df.to_csv("student_scores.csv", index=False)
df_loaded = pd.read_csv("student_scores.csv")
print(df_loaded)
```

Question 5

Install and load required packages for the course.

Solution:

```
pip install numpy pandas scipy matplotlib seaborn statsmodels scikit-learn
import numpy as np, pandas as pd, scipy.stats as stats
import matplotlib.pyplot as plt, seaborn as sns
import statsmodels.api as sm
from sklearn.linear_model import LinearRegression, LogisticRegression
print("All packages loaded.")
```

PRACTICAL 2: DESCRIPTIVE STATISTICS AND VISUALIZATION

Objective: To compute summary statistics by hand and verify with code, then build standard visualizations.

Sample Dataset — Monthly sales (₹'000) of 10 sales reps:

Rep	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10
Sales	42	38	51	29	46	34	61	40	37	44

Sorted: **29, 34, 37, 38, 40, 42, 44, 46, 51, 61**

Question 1

Compute mean, median, and mode.

Mathematical Solution:

Mean: $\bar{x} = \frac{\sum x_i}{n} = \frac{29+34+37+38+40+42+44+46+51+61}{10} = \frac{422}{10} = 42.2$

Median (n=10, even): average of 5th and 6th sorted values $= \frac{40+42}{2} = 41$

Mode: no value repeats → no mode in this sample (dataset is "amodal")

Code Solution:

```
import numpy as np
import pandas as pd

sales = pd.Series([42, 38, 51, 29, 46, 34, 61, 40, 37, 44])

print("Mean:", sales.mean())
print("Median:", sales.median())
print("Mode:", sales.mode().tolist())
```

Interpretation: Mean (42.2) is slightly above the median (41) because R7's high sale of 61 pulls the average up — median is a fairer "typical" value here.

Question 2

Compute variance, standard deviation, and IQR.

Mathematical Solution:

Question 2

Compute variance, standard deviation, and IQR.

Mathematical Solution:

4

Sample variance: $s^2 = \frac{\sum(x_i - \bar{x})^2}{n-1}$

Deviations from mean (42.2): −13.2, −8.2, −5.2, −4.2, −2.2, −0.2, 1.8, 3.8, 8.8, 18.8

Squared deviations: 174.24, 67.24, 27.04, 17.64, 4.84, 0.04, 3.24, 14.44, 77.44, 353.44

Sum = 739.6 → $s^2 = \frac{739.6}{9} = 82.18$

Standard deviation: $s = \sqrt{82.18} = 9.07$

IQR: Q1 = median of lower half (29,34,37,38,40) = 37; Q3 = median of upper half (42,44,46,51,61) = 46
IQR = Q3 − Q1 = 46 − 37 = **9**

Code Solution:

```
variance = sales.var()      # sample variance (ddof=1 by default in pandas)
std_dev = sales.std()
q1 = sales.quantile(0.25)
q3 = sales.quantile(0.75)
iqr = q3 - q1

print(f"Variance: {variance:.2f}")
print(f"Std Dev: {std_dev:.2f}")
print(f"Q1: {q1}, Q3: {q3}, IQR: {iqr}")
```

(Note: pandas' quantile method may give slightly different Q1/Q3 than the manual median-of-halves method due to interpolation — both are valid conventions; this is worth discussing in class.)

Question 3

Compute skewness and kurtosis.

Mathematical Solution:

Sample skewness (Fisher-Pearson, simplified): $g_1 = \frac{\frac{1}{n}\sum(x_i - \bar{x})^3}{s^3}$

Using the cubed deviations of the values above and $s = 9.07$, the numerator is dominated by the +18.8 deviation (R7), cubed = 6644.7, which is much larger in magnitude than the negative deviations cubed — giving a **positive skew** (right-tailed distribution).

Code Solution:

```
skewness = sales.skew()
kurtosis = sales.kurt()
print(f"Skewness: {skewness:.2f}")
print(f"Kurtosis: {kurtosis:.2f}")
```

5

Interpretation: Positive skewness confirms R7's high sale stretches the right tail — manually computing the cubed deviations shows *why*: extreme values get amplified by cubing far more than small deviations, which is exactly what makes skewness sensitive to outliers.

Sample variance: $s^2 = \frac{\sum(x_i - \bar{x})^2}{n-1}$

Deviations from mean (42.2): −13.2, −8.2, −5.2, −4.2, −2.2, −0.2, 1.8, 3.8, 8.8, 18.8

Squared deviations: 174.24, 67.24, 27.04, 17.64, 4.84, 0.04, 3.24, 14.44, 77.44, 353.44

Sum = 739.6 → $s^2 = \frac{739.6}{9} = 82.18$

Standard deviation: $s = \sqrt{82.18} = 9.07$

IQR: Q1 = median of lower half (29,34,37,38,40) = 37; Q3 = median of upper half (42,44,46,51,61) = 46
IQR = Q3 − Q1 = 46 − 37 = **9**

Code Solution:

```
variance = sales.var()      # sample variance (ddof=1 by default in pandas)
std_dev = sales.std()
q1 = sales.quantile(0.25)
q3 = sales.quantile(0.75)
iqr = q3 - q1

print(f"Variance: {variance:.2f}")
print(f"Std Dev: {std_dev:.2f}")
print(f"Q1: {q1}, Q3: {q3}, IQR: {iqr}")
```

(Note: pandas' quantile method may give slightly different Q1/Q3 than the manual median-of-halves method due to interpolation — both are valid conventions; this is worth discussing in class.)

Question 3

Compute skewness and kurtosis.

Mathematical Solution:

Sample skewness (Fisher-Pearson, simplified): $g_1 = \frac{\frac{1}{n}\sum(x_i - \bar{x})^3}{s^3}$

Using the cubed deviations of the values above and $s = 9.07$, the numerator is dominated by the +18.8 deviation (R7), cubed = 6644.7, which is much larger in magnitude than the negative deviations cubed — giving a **positive skew** (right-tailed distribution).

Code Solution:

```
skewness = sales.skew()
kurtosis = sales.kurt()
print(f"Skewness: {skewness:.2f}")
print(f"Kurtosis: {kurtosis:.2f}")
```

Interpretation: Positive skewness confirms R7's high sale stretches the right tail — manually computing the cubed deviations shows *why*: extreme values get amplified by cubing far more than small deviations, which is exactly what makes skewness sensitive to outliers.

Question 4

Build histograms and boxplots.

Procedure (manual boxplot construction first):
Using Q1=37, Median=41, Q3=46, IQR=9:

- Lower whisker = Q1 − 1.5×IQR = 37 − 13.5 = 23.5
- Upper whisker = Q3 + 1.5×IQR = 46 + 13.5 = 59.5
- R7's value of 61 exceeds 59.5 → it is an **outlier** on the boxplot

Code Solution:

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].hist(sales, bins=5, color="steelblue", edgecolor="black")
axes[0].set_title("Histogram of Monthly Sales")
axes[1].boxplot(sales)
axes[1].set_title("Boxplot of Monthly Sales")
plt.tight_layout()
plt.show()
```

Question 5

Build scatter plots and bar charts.

Solution:

```
ad_spend = pd.Series([60, 55, 75, 40, 65, 48, 90, 58, 52, 62])

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].scatter(ad_spend, sales, color="darkgreen")
axes[0].set_title("Ad Spend vs Sales")
axes[1].bar([f"R{i}" for i in range(1,11)], sales, color="coral")
axes[1].set_title("Sales by Rep")
axes[1].tick_params(axis='x', rotation=45)
plt.tight_layout()
plt.show()
```

PRACTICAL 3: PROBABILITY DISTRIBUTIONS

Objective: To compute Binomial, Poisson, and Normal probabilities by hand using their formulas, then verify and visualize using code.

Question 1

Generate Binomial samples and plot the PMF. (n=5 calls, p=0.3 success probability — small n chosen for manual calculation)

Mathematical Solution:

Binomial PMF: $P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$

For n=5, p=0.3, compute P(X=2):

$$P(X = 2) = \binom{5}{2} (0.3)^2 (0.7)^3 = 10 \times 0.09 \times 0.343 = 0.3087$$

Mean of Binomial: $\mu = np = 5 \times 0.3 = 1.5$
Variance: $\sigma^2 = np(1-p) = 5 \times 0.3 \times 0.7 = 1.05$

Code Solution:

```
import numpy as np
import scipy.stats as stats
import matplotlib.pyplot as plt

n, p = 5, 0.3
x = np.arange(0, n+1)
pmf = stats.binom.pmf(x, n, p)

print("P(X=2) from formula check:", stats.binom.pmf(2, n, p))

plt.bar(x, pmf, color="steelblue")
plt.title("Binomial PMF (n=5, p=0.3)")
plt.xlabel("Number of Successes")
plt.show()

print("Mean (n*p):", n*p, "| Variance:", n*p*(1-p))
```

Question 2

Generate Poisson samples and plot the PMF. (λ = 4 complaints/day)

Mathematical Solution:

Poisson PMF: $P(X = k) = \frac{e^{-\lambda}\lambda^k}{k!}$

For λ=4, compute P(X=3):

$$P(X = 3) = \frac{e^{-4} \times 4^3}{3!} = \frac{0.0183 \times 64}{6} = \frac{1.1712}{6} = 0.1954$$

Mean = Variance = λ = 4 (a defining property of the Poisson distribution)

Code Solution:

```
lam = 4
x = np.arange(0, 15)
pmf_poisson = stats.poisson.pmf(x, lam)

print("P(X=3) from formula check:", stats.poisson.pmf(3, lam))

plt.bar(x, pmf_poisson, color="orange")
plt.title("Poisson PMF (λ=4)")
plt.show()
```

Question 3

Generate Normal samples and plot the PDF/CDF. (μ=8 min, σ=2 min — call handling time)

Mathematical Solution:

Normal PDF: $f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$

At x=8 (the mean): $f(8) = \frac{1}{2\sqrt{2\pi}} e^0 = \frac{1}{5.013} = 0.1995$(the peak height of the curve)

Code Solution:

```
mu, sigma = 8, 2
x = np.linspace(0, 16, 200)
pdf = stats.norm.pdf(x, mu, sigma)
cdf = stats.norm.cdf(x, mu, sigma)

print("f(8) from formula check:", stats.norm.pdf(8, mu, sigma))

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].plot(x, pdf, color="purple"); axes[0].set_title("Normal PDF")
axes[1].plot(x, cdf, color="green"); axes[1].set_title("Normal CDF")
plt.tight_layout(); plt.show()
```

Question 4

Compute probabilities for given x values.

Mathematical Solution:

For P(X ≤ 10) using z-score: $z = \frac{x-\mu}{\sigma} = \frac{10-8}{2} = 1.0$

From the standard normal table, $P(Z \leq 1.0) = 0.8413$

So P(X ≤ 10 minutes) ≈ **0.8413**, meaning 84.13% of calls are handled within 10 minutes.

Code Solution:

```
p_leq_10 = stats.norm.cdf(10, mu, sigma)
print(f"P(X <= 10) = {p_leq_10:.4f}")

p_gt_12 = 1 - stats.norm.cdf(12, mu, sigma)
print(f"P(X > 12) = {p_gt_12:.4f}")
```

Question 5

Compute quantiles (z-scores, percentiles).

Mathematical Solution:

z-score for x=11: $z = \frac{11-8}{2} = 1.5$

90th percentile: from the z-table, the z-value where P(Z ≤ z) = 0.90 is **z = 1.28**
Converting back to original scale: $x = \mu + z\sigma = 8 + 1.28(2) = 10.56$minutes

Code Solution:

```
z_score = (11 - mu) / sigma
print(f"Z-score for 11 minutes: {z_score:.2f}")

p90 = stats.norm.ppf(0.90, mu, sigma)
print(f"90th percentile: {p90:.2f} minutes")
```

Interpretation: A call lasting 11 minutes is 1.5 standard deviations above average — useful for flagging unusually long calls. The 90th percentile (~10.56 min) can be used as a service-level benchmark.

PRACTICAL 4: SAMPLING AND CENTRAL LIMIT THEOREM

Objective: To demonstrate, both by hand on a small population and via simulation, that sample means approach a normal distribution as n increases.

Question 1

Define a non-normal population.

Mathematical Setup (small hand-workable population):

Population of 6 transaction values (deliberately skewed): **{10, 20, 20, 30, 40, 200}**

Population mean: $\mu = \frac{10+20+20+30+40+200}{6} = \frac{320}{6} = 53.33$

This population is heavily right-skewed due to the 200 value — a good small example of a non-normal population.

Code Solution:

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
population = np.random.exponential(scale=500, size=100000)  # large skewed population for simulation

plt.hist(population, bins=50, color="gray")
plt.title("Population Distribution (Skewed)")
plt.show()
print("Population mean:", population.mean())
```

Question 2

Draw repeated samples of size n = 10, 30, 100.

Mathematical Illustration (using the small population above, n=2 samples without replacement, by hand):

All possible samples of size 2 from {10,20,20,30,40,200} and their means (a few examples): (10,20)→15, (10,200)→105, (20,30)→25, (40,200)→120 …

Even from this tiny example, you can see sample means vary less wildly than the original values (which ranged from 10 to 200) — this is the seed of CLT.

Code Solution:

```
def draw_sample_means(population, n, num_samples=1000):
```

```
    return [np.random.choice(population, size=n, replace=False).mean() for _ in range(num_samples)]

means_10 = draw_sample_means(population, 10)
means_30 = draw_sample_means(population, 30)
means_100 = draw_sample_means(population, 100)
```

Question 3

Compute sample means.

Mathematical Solution:

Standard Error formula: $SE = \frac{\sigma}{\sqrt{n}}$

If population σ ≈ 500 (matching the exponential scale parameter), then:

- For n=10: $SE = 500/\sqrt{10} = 158.1$
- For n=30: $SE = 500/\sqrt{30} = 91.3$
- For n=100: $SE = 500/\sqrt{100} = 50.0$

This formula predicts the spread will shrink by a factor of $\sqrt{n}$— verify this against the simulated standard deviations below.

Code Solution:

```
print(f"n=10  -> Mean: {np.mean(means_10):.2f}, Std Dev: {np.std(means_10):.2f} (predicted SE: 158.1)")
print(f"n=30  -> Mean: {np.mean(means_30):.2f}, Std Dev: {np.std(means_30):.2f} (predicted SE: 91.3)")
print(f"n=100 -> Mean: {np.mean(means_100):.2f}, Std Dev: {np.std(means_100):.2f} (predicted SE: 50.0)")
```

Question 4

Plot histograms of the sample means.

Code Solution:

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
axes[0].hist(means_10, bins=30, color="lightblue"); axes[0].set_title("n=10")
axes[1].hist(means_30, bins=30, color="lightgreen"); axes[1].set_title("n=30")
axes[2].hist(means_100, bins=30, color="salmon"); axes[2].set_title("n=100")
plt.tight_layout(); plt.show()
```

Question 5

Demonstrate the Central Limit Theorem visually and summarize.

Interpretation: The formula $SE = \sigma/\sqrt{n}$correctly predicts the shrinking spread observed in the histograms — even though individual transactions follow a skewed exponential distribution, the *sample means* become progressively more bell-shaped and tightly clustered

PRACTICAL 5: CONFIDENCE INTERVALS

Objective: To compute confidence intervals for a mean and a proportion by hand using t/z critical values, and verify with code.

Sample Dataset — Product weights (grams, n=10):
248, 251, 247, 253, 250, 249, 252, 246, 254, 250

Question 1

Compute the mean and standard deviation of the sample.

Mathematical Solution:

$$\bar{x} = \frac{248 + 251 + 247 + 253 + 250 + 249 + 252 + 246 + 254 + 250}{10} = \frac{2500}{10} = 250.0$$

Deviations: −2,1,−3,3,0,−1,2,−4,4,0 → squared: 4,1,9,9,0,1,4,16,16,0 → sum = 60

$$s = \sqrt{\frac{60}{9}} = \sqrt{6.67} = 2.58$$

Code Solution:

```
import numpy as np
weights = np.array([248,251,247,253,250,249,252,246,254,250])
print("Mean:", weights.mean())
print("Std Dev:", weights.std(ddof=1))
```

Question 2

Compute 90%, 95%, and 99% confidence intervals for the mean.

Mathematical Solution:

CI formula: $\bar{x} \pm t_{\alpha/2,n-1} \times \frac{s}{\sqrt{n}}$

Standard error: $SE = \frac{2.58}{\sqrt{10}} = 0.816$

For 95% CI, df=9, $t_{0.025,9} = 2.262$(from t-table)

Margin of error = $2.262 \times 0.816 = 1.846$

95% CI = $250 \pm 1.846 = (248.15,251.85)$

For 90% CI, $t_{0.05,9} = 1.833 \rightarrow$ margin $= 1.833 \times 0.816 = 1.496 \rightarrow$ CI = (248.50, 251.50)

For 99% CI, $t_{0.005,9} = 3.250 \rightarrow$ margin $= 3.250 \times 0.816 = 2.652 \rightarrow$ CI = (247.35, 252.65)

Code Solution:

```
import scipy.stats as stats

def confidence_interval(data, confidence):
    n = len(data)
    mean = np.mean(data)
    sem = stats.sem(data)
    margin = sem * stats.t.ppf((1 + confidence) / 2, n - 1)
    return mean - margin, mean + margin

for conf in [0.90, 0.95, 0.99]:
    lower, upper = confidence_interval(weights, conf)
    print(f"{int(conf*100)}% CI: ({lower:.2f}, {upper:.2f})")
```

Interpretation: Notice as confidence level rises (90%→99%), the t-critical value grows and the interval widens — to be "more confident," we must accept a wider range of plausible values.

Question 3

Compute the confidence interval for a proportion. (162 of 200 customers satisfied)

Mathematical Solution:

$$\hat{p} = \frac{162}{200} = 0.81$$

CI formula: $\hat{p} \pm z_{\alpha/2}\sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$

$$SE = \sqrt{\frac{0.81 \times 0.19}{200}} = \sqrt{0.00077} = 0.0277$$

95% CI (z=1.96): margin $= 1.96 \times 0.0277 = 0.0543$

CI $= 0.81 \pm 0.0543 = (0.756, 0.864)$

Code Solution:

```
successes, n_survey = 162, 200
p_hat = successes / n_survey
z_95 = 1.96
margin_p = z_95 * np.sqrt((p_hat * (1 - p_hat)) / n_survey)
print(f"95% CI for proportion: ({p_hat - margin_p:.3f}, {p_hat + margin_p:.3f})")
```

Question 4

Vary sample size and observe how CI width changes.

Mathematical Solution:

CI width $\propto \frac{1}{\sqrt{n}}$. Doubling n does **not** halve the width — it shrinks it by a factor of $\sqrt{2} \approx 1.41$. To actually halve the CI width, n must be **quadrupled**.

Example: width at n=10 is 3.69 (from Q2's 95% CI). At n=40 (4× the sample), predicted width ≈ 3.69/2 = 1.85.

Code Solution:

```
for n_test in [10, 25, 50, 100]:
    subset = np.random.choice(weights, size=min(n_test, len(weights)), replace=True)
    lower, upper = confidence_interval(subset, 0.95)
    print(f"n={n_test}: 95% CI = ({lower:.2f}, {upper:.2f}), Width = {upper-lower:.2f}")
```

Question 5

Interpret the intervals in a business context.

Interpretation: "We are 95% confident the true average product weight lies between 248.15g and 251.85g" — this tells QC whether the batch likely meets its 250g specification. For the satisfaction survey, "we are 95% confident true satisfaction is between 75.6% and 86.4%" — management should not claim a single point figure like "81% satisfied" without acknowledging this range, since the true value could plausibly be as low as ~76%.

PRACTICAL 6: ONE-SAMPLE AND TWO-SAMPLE T-TEST

Objective: To compute t-statistics by hand using the t-test formulas, and verify with code.

Question 1

State the null and alternative hypotheses for a bottle-filling machine claimed to fill 500ml.

Solution (stated):

- **H_0:** $\mu = 500$ml
- **H_1:** $\mu \neq 500$ml (two-tailed test, $\alpha = 0.05$)

Question 2

Perform a one-sample t-test.

Sample Dataset (n=8, ml): 498, 502, 497, 501, 499, 503, 496, 500

Mathematical Solution:

$$\bar{x} = \frac{498 + 502 + 497 + 501 + 499 + 503 + 496 + 500}{8} = \frac{3996}{8} = 499.5$$

Deviations: −1.5, 2.5, −2.5, 1.5, −0.5, 3.5, −3.5, 0.5 → squared sum = 2.25+6.25+6.25+2.25+0.25+12.25+12.25+0.25 = 42

$$s = \sqrt{42/7} = \sqrt{6} = 2.449$$

One-sample t formula: $t = \frac{\bar{x}-\mu_0}{s/\sqrt{n}} = \frac{499.5-500}{2.449/\sqrt{8}} = \frac{-0.5}{0.866} = -0.577$

df = n−1 = 7. From the t-table, critical value $t_{0.025,7} = 2.365$

Since $|-0.577| < 2.365$, we **fail to reject H_0**.

Code Solution:

```
import numpy as np
from scipy import stats

bottles = np.array([498,502,497,501,499,503,496,500])
t_stat, p_value = stats.ttest_1samp(bottles, popmean=500)
print(f"t-statistic: {t_stat:.3f}, p-value: {p_value:.4f}")
```

Question 3

Perform an independent two-sample t-test. (Shift A vs Shift B units produced)

Sample Dataset: Shift A = [45,48,42,50], Shift B = [52,55,50,58]

Mathematical Solution:

$\bar{x}_A = 46.25, \bar{x}_B = 53.75$

s_A^2: deviations −1.25,1.75,−4.25,3.75 → squared: 1.56,3.06,18.06,14.06 → sum=36.74 → $s_A^2 = 36.74/3 = 12.25$

s_B^2: deviations −1.75,1.25,−3.75,4.25 → squared: 3.06,1.56,14.06,18.06 → sum=36.74 → $s_B^2 = 36.74/3 = 12.25$

Pooled variance: $s_p^2 = \frac{(n_A-1)s_A^2+(n_B-1)s_B^2}{n_A+n_B-2} = \frac{3(12.25)+3(12.25)}{6} = 12.25$

$$t = \frac{\bar{x}_A - \bar{x}_B}{\sqrt{s_p^2(\frac{1}{n_A}+\frac{1}{n_B})}} = \frac{46.25-53.75}{\sqrt{12.25(0.25+0.25)}} = \frac{-7.5}{\sqrt{6.125}} = \frac{-7.5}{2.475} = -3.03$$

df = $n_A + n_B - 2 = 6$; critical value $t_{0.025,6} = 2.447$. Since $|-3.03| > 2.447$, **reject H_0** — Shift B is significantly more productive.

Code Solution:

```
shift_a = np.array([45, 48, 42, 50])
shift_b = np.array([52, 55, 50, 58])
t_stat, p_value = stats.ttest_ind(shift_a, shift_b)
print(f"t-statistic: {t_stat:.3f}, p-value: {p_value:.4f}")
```

Question 4

Perform a paired t-test. (Employee scores before vs after training)

Sample Dataset: Before=[65,70,60,75], After=[72,75,68,80]

Mathematical Solution:

Differences (After−Before): 7, 5, 8, 5

$$\bar{d} = \frac{7+5+8+5}{4} = \frac{25}{4} = 6.25$$

Deviations from $\bar{d}$: 0.75, −1.25, 1.75, −1.25 → squared: 0.5625, 1.5625, 3.0625, 1.5625 → sum = 6.75

$$s_d = \sqrt{6.75/3} = \sqrt{2.25} = 1.5$$

$$t = \frac{\bar{d}}{s_d/\sqrt{n}} = \frac{6.25}{1.5/2} = \frac{6.25}{0.75} = 8.33$$

df=3; critical value $t_{0.025,3} = 3.182$. Since 8.33 > 3.182, **reject H₀** — training significantly improved scores.

Code Solution:

```
before = np.array([65, 70, 60, 75])
after = np.array([72, 75, 68, 80])
t_stat, p_value = stats.ttest_rel(before, after)
print(f"t-statistic: {t_stat:.3f}, p-value: {p_value:.4f}")
```

Question 5

Interpret the p-values from all three tests and conclude.

Test	Manual t	Conclusion
One-sample (bottle fill)	−0.577 (not significant)	Machine output is consistent with 500ml claim
Two-sample (Shift A vs B)	−3.03 (significant)	Shift B significantly outproduces Shift A
Paired (before vs after)	8.33 (significant)	Training significantly improved performance

General Rule: Reject H₀ when the computed |t| exceeds the critical t-value for the chosen α and degrees of freedom (equivalently, when p-value < α).

PRACTICAL 7: CHI-SQUARE TEST FOR INDEPENDENCE

Objective: To compute the chi-square statistic by hand from a contingency table, and verify with code.

Sample Dataset — Contingency Table (Gender × Shopping Channel):

	Online	In-Store	Total
Male	30	20	50
Female	20	30	50
Total	50	50	100

Question 1

Build the contingency table.

Code Solution:

```
import pandas as pd
observed = pd.DataFrame({"Online": [30, 20], "InStore": [20, 30]}, index=["Male", "Female"])
print(observed)
```

Question 2

Compute expected frequencies.

Mathematical Solution:

Formula: $E_{ij} = \frac{(\text{Row Total}) \times (\text{Column Total})}{\text{Grand Total}}$

$$E_{Male,Online} = \frac{50 \times 50}{100} = 25$$
$$E_{Male,InStore} = \frac{50 \times 50}{100} = 25$$
$$E_{Female,Online} = \frac{50 \times 50}{100} = 25$$
$$E_{Female,InStore} = \frac{50 \times 50}{100} = 25$$

(All expected = 25 since row and column totals are equal in this example.)

Code Solution:

```
from scipy.stats import chi2_contingency
chi2_stat, p_value, dof, expected = chi2_contingency(observed)
print("Expected Frequencies:\n", expected)
```

Question 3

Compute the chi-square statistic.

Mathematical Solution:

$$\chi^2 = \sum \frac{(O-E)^2}{E}$$

$$= \frac{(30-25)^2}{25} + \frac{(20-25)^2}{25} + \frac{(20-25)^2}{25} + \frac{(30-25)^2}{25}$$

$$= \frac{25}{25} + \frac{25}{25} + \frac{25}{25} + \frac{25}{25} = 1 + 1 + 1 + 1 = 4.0$$

df = (rows−1)(cols−1) = (2−1)(2−1) = 1

Code Solution:

```
print(f"Chi-Square statistic: {chi2_stat:.3f}")
print(f"Degrees of freedom: {dof}")
```

Question 4

Compare with the critical value / p-value.

Mathematical Solution:

From the chi-square table, critical value at α=0.05, df=1 is **3.841**.

Since $\chi^2_{calc} = 4.0 > 3.841$, we **reject H_0** — there is a significant association.

Code Solution:

```
from scipy.stats import chi2
critical_value = chi2.ppf(0.95, dof)
print(f"Critical value: {critical_value:.3f}")
print(f"p-value: {p_value:.4f}")
```

Question 5

Interpret the association in a business context.

PRACTICAL 8: ONE-WAY ANOVA

Objective: To compute the F-statistic by hand using between- and within-group variance, and verify with code.

Sample Dataset — Daily sales (₹'000) by store layout, 3 stores per layout:

Layout A	Layout B	Layout C
42	58	50
45	60	52
48	62	54

Question 1

Prepare the dataset with 3+ groups.

Code Solution:

```
import pandas as pd
data = pd.DataFrame({
    "Sales": [42,45,48, 58,60,62, 50,52,54],
    "Layout": ["A"]*3 + ["B"]*3 + ["C"]*3
})
print(data)
```

Question 2

Check assumptions (normality, equal variance).

Code Solution:

```
from scipy import stats
a = data[data["Layout"]=="A"]["Sales"]
b = data[data["Layout"]=="B"]["Sales"]
c = data[data["Layout"]=="C"]["Sales"]
levene_stat, levene_p = stats.levene(a, b, c)
print(f"Levene's p-value: {levene_p:.4f}")
```

Question 3

Perform one-way ANOVA.

Mathematical Solution:

Group means: $\bar{x}_A = 45, \bar{x}_B = 60, \bar{x}_C = 52$. Grand mean: $\bar{x} = \frac{45+60+52}{3} = 52.33$

Between-group sum of squares (SSB):

$$SSB = n\sum(\bar{x}_i - \bar{x})^2 = 3[(45 - 52.33)^2 + (60 - 52.33)^2 + (52 - 52.33)^2]$$
$$= 3[53.71 + 58.83 + 0.11] = 3(112.65) = 337.96$$

Within-group sum of squares (SSW) — sum of squared deviations within each group:
Group A: $(42-45)^2+(45-45)^2+(48-45)^2 = 9+0+9 = 18$
Group B: $(58-60)^2+(60-60)^2+(62-60)^2 = 4+0+4 = 8$
Group C: $(50-52)^2+(52-52)^2+(54-52)^2 = 4+0+4 = 8$

$$SSW = 18 + 8 + 8 = 34$$

$df_{between} = k - 1 = 2, df_{within} = N - k = 9 - 3 = 6$

$$MSB = SSB/df_{between} = 337.96/2 = 168.98$$
$$MSW = SSW/df_{within} = 34/6 = 5.67$$

$$F = \frac{MSB}{MSW} = \frac{168.98}{5.67} = 29.8$$

Code Solution:

```
f_stat, p_value = stats.f_oneway(a, b, c)
print(f"F-statistic: {f_stat:.3f}, p-value: {p_value:.5f}")
```

Question 4

Interpret the F-statistic and p-value.

Mathematical Solution:

Critical F-value at α=0.05, df1=2, df2=6 is **5.14** (from F-table).

Since $F_{calc} = 29.8 > 5.14$, we **reject H_0** — at least one layout has a significantly different mean.

Question 5

Conduct a post-hoc test (Tukey HSD).

Code Solution:

```
from statsmodels.stats.multicomp import pairwise_tukeyhsd
tukey_result = pairwise_tukeyhsd(endog=data["Sales"], groups=data["Layout"], alpha=0.05)
print(tukey_result)
```

PRACTICAL 9: SIMPLE AND MULTIPLE LINEAR REGRESSION

Objective: To compute regression coefficients by hand using the least-squares formulas, and verify with code.

Sample Dataset — TV Ad Spend (₹'000) vs Sales (₹'000), n=5:

TV_Spend (x)	Sales (y)
100	150
150	200
200	230
250	280
300	320

Question 1

Load the advertising dataset.

Code Solution:

```
import pandas as pd
ads = pd.DataFrame({"TV_Spend": [100,150,200,250,300], "Sales": [150,200,230,280,320]})
print(ads)
```

Question 2

Fit a simple linear regression by hand: Sales = b_0 + b_1(TV_Spend)

Mathematical Solution:

$$\bar{x} = \frac{100+150+200+250+300}{5} = 200,\ \bar{y} = \frac{150+200+230+280+320}{5} = 236$$

Slope: $b_1 = \frac{\sum(x_i-\bar{x})(y_i-\bar{y})}{\sum(x_i-\bar{x})^2}$

$x-\bar{x}$	$y-\bar{y}$	$(x-\bar{x})(y-\bar{y})$	$(x-\bar{x})^2$
−100	−86	8600	10000

$x-\bar{x}$	$y-\bar{y}$	$(x-\bar{x})(y-\bar{y})$	$(x-\bar{x})^2$
−50	−36	1800	2500
0	−6	0	0
50	44	2200	2500
100	84	8400	10000

Sum: numerator = 21000, denominator = 25000

$$b_1 = \frac{21000}{25000} = 0.84$$

Intercept: $b_0 = \bar{y} - b_1\bar{x} = 236 - 0.84(200) = 236 - 168 = 68$

Regression equation: Sales = 68 + 0.84 × TV_Spend

Code Solution:

```
import statsmodels.api as sm
X_simple = sm.add_constant(ads["TV_Spend"])
simple_model = sm.OLS(ads["Sales"], X_simple).fit()
print(simple_model.params)
```

Question 3

Fit a multiple linear regression with an added Online_Spend variable.

Code Solution:

```
ads["Online_Spend"] = [20, 25, 35, 45, 55]
X_multi = sm.add_constant(ads[["TV_Spend", "Online_Spend"]])
multi_model = sm.OLS(ads["Sales"], X_multi).fit()
print(multi_model.summary())
```

(Manual computation of multiple regression involves matrix algebra: $b = (X^T X)^{-1} X^T y$— beyond hand calculation at this level; students should understand the simple regression math above conceptually extends here.)

Question 4

Interpret the coefficients and R^2.

Mathematical Solution:

$$R^2 = \frac{SSR}{SST} = 1 - \frac{SSE}{SST}$$

Predicted values using Sales = 68 + 0.84x:
x=100→152, x=150→194, x=200→236, x=250→278, x=300→320

Residuals (actual−predicted): 150−152=−2, 200−194=6, 230−236=−6, 280−278=2, 320−320=0

$$SSE = (-2)^2 + 6^2 + (-6)^2 + 2^2 + 0^2 = 4 + 36 + 36 + 4 + 0 = 80$$

$$SST = \sum(y_i - \bar{y})^2 = (-86)^2 + (-36)^2 + (-6)^2 + 44^2 + 84^2$$
$$= 7396 + 1296 + 36 + 1936 + 7056 = 17720$$

$$R^2 = 1 - \frac{80}{17720} = 1 - 0.0045 = 0.9955$$

Code Solution:

```
print(f"Simple model R²: {simple_model.rsquared:.4f}")
```

Interpretation: b_1=0.84 means every additional ₹1,000 in TV ad spend is associated with ₹840 more in sales. R^2=0.9955 means TV spend alone explains 99.55% of the variation in sales in this sample — an unusually strong fit typical of small, clean illustrative datasets.

Question 5

Plot residuals and assess model fit.

Code Solution:

```
import matplotlib.pyplot as plt
fitted_vals = simple_model.fittedvalues
residuals = simple_model.resid

plt.scatter(fitted_vals, residuals, color="teal")
plt.axhline(0, color="red", linestyle="--")
plt.title("Residuals vs Fitted Values")
plt.show()
```

Interpretation: Residuals scattered randomly around zero with no funnel or curved pattern support the linear model's validity; a clear pattern would suggest a non-linear relationship or violated assumptions.

PRACTICAL 10: LOGISTIC REGRESSION MINI PROJECT

Objective: To understand the logistic function mathematically, then build and evaluate a logistic regression model in code.

Question 1

Load a binary classification dataset (loan default).

Sample Dataset:

Income('000)	CreditScore	Defaulted
45	620	1
80	740	0
32	590	1
95	760	0
50	640	1
70	710	0

Code Solution:

```
import pandas as pd
loan = pd.DataFrame({
   "Income": [45,80,32,95,50,70],
   "CreditScore": [620,740,590,760,640,710],
   "Defaulted": [1,0,1,0,1,0]
})
print(loan)
```

Question 2

Split into train and test sets.

Code Solution:

```
from sklearn.model_selection import train_test_split
X = loan[["Income", "CreditScore"]]
y = loan["Defaulted"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_state=42)
```

Question 3

Fit a logistic regression model.

Mathematical Solution (the logistic function):

Unlike linear regression, logistic regression models the **log-odds** as linear in x:

$$\ln\left(\frac{p}{1-p}\right) = b_0 + b_1 x$$

Solving for p gives the **sigmoid function**:

$$p = \frac{1}{1 + e^{-(b_0 + b_1 x)}}$$

Worked example: Suppose after fitting, $b_0 = 12$ and b_1(CreditScore) $= -0.018$ (illustrative values). For an applicant with CreditScore=650:

$$p = \frac{1}{1 + e^{-(12+(-0.018)(650))}} = \frac{1}{1 + e^{-(12-11.7)}} = \frac{1}{1 + e^{-0.3}} = \frac{1}{1 + 0.741} = \frac{1}{1.741} = 0.574$$

So this applicant has an estimated 57.4% probability of default — above the typical 0.5 threshold, so the model would classify them as "Default: Yes."

Code Solution:

```
from sklearn.linear_model import LogisticRegression
log_model = LogisticRegression()
log_model.fit(X_train, y_train)
print("Coefficients:", log_model.coef_)
print("Intercept:", log_model.intercept_)
```

Question 4

Compute the confusion matrix, accuracy, precision, and recall.

Mathematical Solution:

Given a confusion matrix:

	Predicted No	**Predicted Yes**
Actual No	TN	FP
Actual Yes	FN	TP

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision $= \frac{TP}{TP+FP}$(of those predicted to default, how many actually did)

Recall $= \frac{TP}{TP+FN}$(of those who actually defaulted, how many were caught)

Code Solution:

```
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score

y_pred = log_model.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")
print(f"Precision: {precision_score(y_test, y_pred, zero_division=0):.2f}")
print(f"Recall: {recall_score(y_test, y_pred, zero_division=0):.2f}")
```

Question 5

Prepare and present the final report.

Report Outline:

1. **Problem Statement:** Predicting loan default risk from Income and CreditScore.
2. **Methodology:** Logistic regression using the sigmoid link function; train-test split for evaluation.
3. **Manual Worked Example:** Show the sigmoid calculation for one applicant (as in Question 3) so the audience sees how a probability is derived, not just a label.
4. **Performance Metrics:** Report Accuracy, Precision, Recall, Confusion Matrix.
5. **Business Recommendation:** Applicants with predicted default probability above 50% should be flagged for manual underwriting review.
6. **Limitations:** Tiny sample size here is for teaching; production use requires thousands of records and feature scaling/regularization.

Conclusion: Understanding the sigmoid transformation behind logistic regression — not just calling .fit() — equips students to explain *why* the model outputs probabilities bounded between 0 and 1, which is essential for justifying risk-based decisions to non-technical stakeholders like loan officers.